Verification bot: what changed, how to test it, how to put it back online

12 October 2026. Written for the owner of the /r/sysadmin Discord server.

This guide compares the new bot (branch `claude-refactor`, pull request #36) with the old single-file bot `verif.py` that production was running. It is long because it is meant to be complete. You do not need to read it in one go. Section 1 is the summary and your to-do list. Sections 8 to 11 are the steps you actually carry out.

1. Summary

What happened

The bot has been offline since about 9 October 2026. New members cannot verify. Two things caused this.

1. **Discord took away the bot's special permissions.** Discord calls these permissions "privileged intents". An intent is a switch that decides which kinds of events Discord sends to a bot (for example "a member joined"). Since 10 June 2026, a bot that reaches 10,000 or more users must pass a review to keep its privileged intents. There was a 90-day window to apply, and it was missed. After that, Discord removes the intents. The old bot asked for every intent and could not start without them. That this removal is what started the outage is likely, but it has not been confirmed.
2. **The server restarted the bot every second, forever.** The program that keeps the bot running on the server (the "systemd service", explained in section 7) was set to restart the bot one second after any failure, with no limit. Discord allows a bot 1000 connections in 24 hours. Past that, Discord cancels the bot's token (its password). That happened. The service then tried 16,304 more times with the dead token.

What the new bot is

- It asks Discord for one privileged intent only: **Server Members** (seeing members join and boost). It no longer reads any message text and no longer sees who is online.
- If Discord refuses even that one intent, the bot still starts, in a **reduced mode**: every command works, only the automatic parts are off. It cannot die from a refused intent any more.
- It cannot loop any more. The new service waits 30 seconds, tries at most 5 times an hour, and does not retry at all after a problem that retrying cannot fix (bad token, bad config file, refused intent).
- Every command is now a **slash command**: you type `/` and pick the command from Discord's menu. Members verify with `/verify` instead of sending the word "verify" in a direct message (DM).
- Administrators change the bot's settings from Discord with `/config`. Your existing `config.json` is imported automatically. You retype nothing.

Your to-do list, in order

1. Run the click-through checklist in the **test server**, twice (section 8).
2. Commit, push and merge pull request #36. Nothing has been committed for you.
3. In the Discord Developer Portal: get a new token, switch off the two intents the bot no longer uses, set the privacy policy link (section 10).
4. Deploy on the server `cannizzaro` (section 9).
5. Submit the intent application to Discord, using `docs/discord-intent-application.md` (section 10).

6. When Discord approves: restart the bot once, and set a calendar reminder, because the review repeats every year (section 11).

Until step 6 is done, production will most likely run in reduced mode. That is fine: `/verify` works in reduced mode, so members can get in again as soon as step 4 is done.

2. What changed for members, moderators and administrators

For members

- **Verifying.** Old: send the bot a DM containing "verify". New: run `/verify`. It works in a DM with the bot and in any channel of the server. The answer is private: only the person who ran it sees it.
- **Sending the bot a DM.** The bot no longer reads DMs. It answers any DM with a short pointer to `/verify`, at most once every 10 minutes per person.
- **The text under the bot's name** is now "/verify to get access" instead of "message me to get verified!".
- **Old commands such as `+ping` do nothing now.** The bot cannot see message text, so it cannot see them. The slash commands replace them.
- ****`/emergency` asks for a reason** (20 to 500 characters) and uses buttons instead of reactions. Details in section 6.
- **Two new commands:** `/help` lists every command, and `/privacy` says what the bot does with people's data. Both answers are private.

For moderators

"Moderator" means: someone holding one of the "Moderator roles" from the settings. New: anyone with Discord's Administrator permission also counts as a moderator.

- `/clearemergency` and `/irc` replace `+clearemergency` and `+irc`.
- `/irc` answers are now private to the moderator who ran the command. The record still goes to the mod-log. The **mod-log** is the private channel where the bot records what it did (the old bot called it the log channel).
- You vote on emergency requests with Approve and Deny buttons, not reactions.
- The mod-log now also shows problems you need to act on, such as "could not assign the verified role", with the real member mention. It also shows every settings change.
- The `+birthday` command is gone.
- `/irc` and `/clearemergency` are shown to everyone in Discord's command menu, but only moderators can run them: anyone else gets a private "You do not have permission" reply. This is the same rule as in the old bot (the moderator roles decide), plus administrators. If you would rather hide them from regular members, you can do that in Server Settings > Integrations > the bot.

For administrators

"Administrator" means: someone with Discord's Administrator permission in the server.

- `/config` shows and changes every setting, from inside Discord (section 3).
- `config.json` on the server now only needs three things: the token, the server ID and the name of the IRC relay container (section 5).
- The bot runs from a new service file with strict restart rules (section 7), and logs to the service log instead of a file.
- The bot has two modes, full and reduced (section 7).

Old command to new command

Old	New	Who may use it	What is different
DM the word "verify"	`/verify`	Everyone	Slash command. Works in a DM and in the server. Private answer. At most 2 tries per minute per person.
`+steve`	`/steve`	Everyone, in the server	Same output.
`+ping`	`/ping`	Everyone, in the server	Same idea: shows the bot's response time.
`+uptime`	`/uptime`	Everyone, in the server	Also shows since when the bot has been online.
`+github`	`/github`	Everyone, in the server	Same link.
`+boosters`	`/boosters`	Everyone, in the server	A numbered list, longest-boosting first. In reduced mode it answers "temporarily unavailable".
`+emergency`	`/emergency reason:...`	Verified members, in the on-topic category	A reason is required. Buttons replace reactions. Details in section 6.
`+clearemergency`	`/clearemergency`	Moderators	Same job: cancels the pending request.
`+irc start` (or stop, restart, status)	`/irc action:...`	Moderators	The action is picked from a list. The answer is private. Only exists when `irc_relay_id` is set in `config.json`.
`+birthday @member`	(removed)	-	Removed. See section 4.
(none)	`/help`	Everyone	New. Lists every command, grouped as Everyone, Verified members, Moderators, Administrators. Private answer.
(none)	`/privacy`	Everyone	New. Short summary, link to the privacy policy, how to ask for deletion. Private answer.
(none)	`/config ...`	Administrators, in the server	New. Shows and changes settings. Section 3.

`/verify`, `/help` and `/privacy` are registered "globally", which means they also work in a DM with the bot. All the others exist only inside the /r/sysadmin server.

The waiting times between uses are the same as before: 10 seconds per person for `/steve`, `/ping`, `/uptime`, `/github` (and for `/help` and `/privacy`), 15 seconds per channel for `/boosters`, 2 minutes for the whole server for `/emergency`.

3. The /config command

`/config` is what you use to look at the bot's settings and change them without touching a file on the server. A "setting" is a value the bot needs to do its job: which role to give, which channel to write to, how many approvals an emergency needs.

Rules that apply to every `/config` command:

- Only administrators can use it, and only inside the /r/sysadmin server.
- You never type an ID. You pick the setting from a list, and the role, channel or user from Discord's own picker.
- The answer is private to you.

- A change takes effect immediately. No restart.
- Every change is posted in the mod-log: who changed which setting, the value before and the value after.
- The values are saved in a file the bot writes itself, `data/settings.json`. It holds role IDs, channel IDs, numbers and the status text. It holds nothing about any member.

The subcommands

Command	What it is for	Example
`/config show`	Lists every setting with its value. Marks every single role, channel, category or user that is not set, and anything pointing at a role or channel that no longer exists, with "Needs attention". An empty list of roles is shown as "none chosen" without a mark, because an empty list is allowed. Also says whether the bot is in full or reduced mode.	`/config show`
`/config role`	Chooses the role for a setting that holds one role.	`/config role setting:Verified role role:@Verified`
`/config channel`	Chooses the text channel for a setting that holds a channel.	`/config channel setting:Mod-log channel channel:#mod-log`
`/config category`	Chooses the channel category (the collapsible group of channels) for the on-topic setting.	`/config category setting:On-topic category category:On-Topic`
`/config user`	Chooses the user for the Moderator Mail setting.	`/config user setting:Moderator Mail user user:@Moderator Mail`
`/config number`	Changes a number. The allowed range is shown next to each name.	`/config number setting:Emergency approval threshold value:3`
`/config status`	Changes the "Playing ..." text under the bot's name (at most 128 characters).	`/config status text:/verify to get access`
`/config role-add`	Adds one role to a setting that holds a list of roles.	`/config role-add setting:Sudo voter roles role:@sudo1`
`/config role-remove`	Takes one role out of such a list.	`/config role-remove setting:Moderator roles role:@Helper`
`/config role-prune`	Removes from a list every role that was deleted from the server.	`/config role-prune setting:Emergency bypass roles`
`/config clear`	Unsets a setting. A number or the status text goes back to its default. A role, channel or user goes back to "not set".	`/config clear setting:Unboost channel`

The bot refuses a few choices and tells you why: `@everyone` is never accepted, and a role managed by Discord or another bot cannot be used as the verified role, the booster role or a colour role. It also warns you (but still saves) when a role it must hand out sits above the bot's own role, and when it cannot post in the mod-log channel you picked.

All settings

There are 21 settings. "Not set" means the bot has no value until you choose one or the import fills it in.

Setting (label in Discord)	What it is for	Default
Verified role	The role the bot gives to a member once they are verified.	Not set
Moderator roles	Roles that may use the moderator commands (`/clearemergency`, `/irc`). A list.	Empty list

Setting (label in Discord)	What it is for	Default
Sudo voter roles	Roles that may approve or deny an emergency request. A list. While it is empty, anyone whose top role is above the verified role may vote.	Empty list
Emergency bypass roles	Roles whose `/emergency` pings straight away, with no vote. A list.	Empty list
Emergency role	The role that gets pinged when an emergency is approved.	Not set
Booster role	The role given to members while they boost the server.	Not set
Colour roles	Booster-only colour roles, taken away when a member stops boosting. A list.	Empty list
Mod-log channel	The private channel where the bot records what it did and what went wrong.	Not set
Unboost channel	The channel where the bot announces that a member stopped boosting.	Not set
Rules channel	Linked in the message a member gets after verifying.	Not set
Roles channel	The self-assignable roles channel, linked in the verification and boost messages.	Not set
Booster channel	The boosters-only channel, linked in the boost thank-you message.	Not set
On-topic category	The channel category in which `/emergency` may be used.	Not set
Moderator Mail user	The mod-mail bot that members are told to contact. While it is not set, the bot's owner is named instead.	Not set
Join age threshold (days)	Accounts older than this many days are verified automatically when they join. Allowed: 0 to 36500.	90
/verify age threshold (days)	Accounts must be at least this many days old for `/verify` to work. Allowed: 0 to 36500.	5
Emergency approval threshold	How many approvals an emergency request needs. Allowed: 1 to 25.	3
Emergency denial threshold	How many denials reject an emergency request. Allowed: 1 to 25.	2
Emergency reminder interval (seconds)	How long the bot waits between reminders about a pending emergency request. Allowed: 15 to 3600.	180
Emergency reminder count	How many reminders are sent before a pending emergency request times out. Allowed: 1 to 24.	5
Status text	The "Playing ..." text shown under the bot's name.	/verify to get access

On the production server these defaults hardly matter, because the first start imports your real values (section 5).

What happens while a setting is not set:

- No verified role: `/verify` and `/emergency` answer that an administrator has to run `/config`. Nobody is auto-verified on join.
- No emergency role or no on-topic category: `/emergency` answers that an administrator has to run `/config`.
- No mod-log channel: the bot keeps working but records nothing in Discord. `/config` tells you so after each change.
- No rules, roles or booster channel: the messages say "the rules channel" and so on in plain words instead of a link.

4. What was removed and why

Several of these things never ran in production. They were added earlier on the `claude-refactor` branch and you decided you did not want them. They are listed so that nothing in the pull request surprises you.

Removed	What it was	Why it is gone
`Dockerfile`, `docker-compose.yml`, `.dockerignore`	Files for running the bot inside a Docker container. Only on the refactor branch.	You run the bot directly on the server with systemd. Nothing used these files.
`secrets/` folder	A place to keep the token and a log key as files for the Docker setup. Only on the refactor branch.	It belonged to the Docker setup. The token lives in `config.json` (or the `DISCORD_TOKEN` environment variable).
Encrypted log file and its key	The refactor branch wrote a log file to disk in scrambled form, and needed a secret key to read it back.	The bot now writes no log file at all, so there is nothing to scramble and no key to keep safe. The tool for reading that file (`logtools.py`) and the extra software package it needed (`cryptography`) are gone too.
Heartbeat file	A small file the bot rewrote every so often so Docker could tell it was alive. Only on the refactor branch.	Only Docker used it. systemd watches the process itself.
The old log file `verification.log`	The old bot wrote every action to this file, with usernames, user IDs and the full text of every DM it received.	It kept personal data outside Discord. The new bot logs only to the service log, with scrambled IDs (section 7).
`+birthday`	A moderator command that gave a member the birthday role and took it back after 18 hours.	You decided to drop it. The `birthday_role_ID` key in `config.json` is ignored.
Prefix commands (`+something`)	Commands the bot recognised by reading every message and looking for a leading `+`.	Reading messages needs the Message Content intent, which the bot no longer has. Slash commands need no intent.
Reading DM text	The old bot read each DM to look for the word "verify".	Same reason: it needs Message Content. `/verify` replaces it. The bot still notices that a DM arrived (so it can answer with the pointer), but it never looks at the text.
Presence intent	The permission to see who is online and what they are doing. The old bot asked for it but never used it.	Not needed. Asking for it would make the Discord review harder.
Message Content intent	The permission to read the text of messages.	Not needed any more, see the two rows above.
Reaction voting	Emergency votes by adding a reaction to the bot's message.	Replaced by buttons, which work without any intent and let the bot answer each voter privately.
Hard-coded channel IDs	The rules, roles and booster channel IDs were typed into the old code.	They are settings now (section 5).

5. Where each old config.json key lives now

The one-time import

The first time the new bot starts, it notices that there is no settings file yet. It then reads your existing `config.json`, exactly as it is, copies every value it understands into `data/settings.json`, and carries on. You retype nothing and you do not need to edit the old file first.

Things to know about it:

- It happens once. On later starts the bot reads `data/settings.json` and ignores the old keys in `config.json`. You may leave them there or delete them.
- The log says so: `Imported 20 setting(s) from config.json into data/settings.json`. On production the number should be 20 if every value in your file is filled in: 17 from the file plus the three channels mentioned below.
- A value the bot cannot use (not a number, or outside the allowed range) is skipped. The bot still starts, the setting keeps its default, and the log has a line starting with "Skipped". `/config show` lets you check and fix it.
- If you ever delete `data/settings.json`, the next start imports from `config.json` again. Changes made with `/config` since then would be lost. So: do not delete it, and include it if you back up the server.
- A brand new install with nothing to import also starts. Commands that need a setting then say that an administrator has to run `/config`.

The table

Old key in `config.json`	**New home**	**Notes**
`token`	Stays in `config.json`	Needs the NEW token (section 10). Can instead be given in the `DISCORD_TOKEN` environment variable, which wins over the file if both are set.
`homeserver_id`	Stays in `config.json`	The ID of the /r/sysadmin server.
`irc_relay_id`	Stays in `config.json`	May be the container's name or its ID. The name is better (section 9). Empty or missing means there is no `/irc` command.
`verified_role`	`/config`: Verified role	
`moderator_role_IDs`	`/config`: Moderator roles	A list, as before.
`emergency_top_role_bypass_id`	`/config`: Emergency bypass roles	Was a single value, is now a list of roles. Your single value becomes a list with one role. See section 6: this now actually works.
`emergency_role_id`	`/config`: Emergency role	
`nitro_role`	`/config`: Booster role	
`colored_roles`	`/config`: Colour roles	A list, as before.
`log_channel_id`	`/config`: Mod-log channel	
`unboost_announcement_channel_id`	`/config`: Unboost channel	
`ontopic_category_id`	`/config`: On-topic category	
`moderator_mail_id`	`/config`: Moderator Mail user	
`verification_requirement_join`	`/config`: Join age threshold (days)	
`verification_requirement_message`	`/config`: /verify age threshold (days)	

Old key in `config.json`	New home	Notes
`emergency_request_approval_threshold`	`/config`: Emergency approval threshold	
`emergency_request_deny_threshold`	`/config`: Emergency denial threshold	
`emergency_role_reminder`	`/config`: Emergency reminder interval (seconds)	Seconds between reminders. Must be 15 to 3600 to be imported.
`emergency_role_timeout`	`/config`: Emergency reminder count	Despite its old name this was always a count of reminders, not a time. Must be 1 to 24 to be imported.
`status`	`/config`: Status text	The old text "message me to get verified!" is wrong now, so it is replaced by "/verify to get access". Any other text is kept.
`prefix`	Ignored	There are no prefix commands any more.
`birthday_role_ID`	Ignored	The birthday command is gone.
`log_directory`, `log_retention_days`	Ignored	These only existed on the refactor branch, for the log file that was removed.
(not in the file: rules channel)	`/config`: Rules channel	The old code had the ID `448941767325253632` typed into it. Imported automatically, but only when `homeserver_id` is the production server.
(not in the file: roles channel)	`/config`: Roles channel	Same, with `516816975427797012`.
(not in the file: booster channel)	`/config`: Booster channel	Same, with `585867795942604800`.
(new)	`/config`: Sudo voter roles	Did not exist before. Starts empty, which keeps the old rule for who may vote.

The three channel IDs are filled in only for the production server (ID `362843193617678337`). In any other server, such as your test server, those three settings start as "not set". If your `config.json` ever contains `rules_channel_id`, `roles_channel_id` or `booster_channel_id`, those values are used instead.

6. Behaviour changes to expect

Emergency requests

The bypass now works. A bypass role is a role whose holders can ping the emergency role immediately, without a vote. In the old bot this never worked. The config held the role as one piece of text, for example "123456", and the old code went through that text one character at a time, ending up with the "roles" 1, 2, 3, 4, 5 and 6. No such roles exist, so nobody ever matched. The new bot reads the value correctly. From the first start on production, holders of that role will ping with no vote. You said you want this. Check with `/config show` that the role listed under "Emergency bypass roles" is the one you expect.

A reason is required. `/emergency` will not run without a `reason` of 20 to 500 characters. The reason is shown in the request, in the final ping and in the mod-log. It is never written to the service log. Anything in the reason that looks like a mention (@someone, @everyone, a role) is defused so it cannot ping anybody.

Buttons replace reactions. The request carries two buttons, Approve and Deny. When someone presses one, the bot answers that person privately ("Your vote to approve has been recorded") and updates the count on the request. Each person has one vote and may change it by pressing the other button. In the old bot the same person could be counted more than once.

The request is a normal message, and its buttons are removed when it ends. The person who ran `/emergency` gets a short private confirmation with a link. The request itself is an ordinary message in the channel, posted by the bot. However it ends (approved, denied, cancelled, timed out, or the bot being stopped), the bot edits that message to say "This request is closed" with the outcome, and takes the buttons away. An earlier version on the refactor branch could leave dead buttons behind after 15 minutes. That is fixed.

Who may vote. Members holding one of the "Sudo voter roles". That setting is new and starts empty. While it is empty, the old rule applies: anyone whose highest role is above the verified role. The person who made the request can never vote on it. Someone who may not vote and presses a button gets a private "Only members of sudo level 1 or higher can vote on emergency requests."

Timing. Same as before. With the default values (180 seconds, 5 reminders):

Minutes after the request	What the bot does
3, 6, 9, 12, 15	Posts a reminder in the channel with a link to the request. Reminders ping nobody.
18	Times the request out: says so in the channel, closes the request message, records it in the mod-log.

In general the request lives for (reminder count + 1) times the reminder interval.

Other details.

- Only one request can be open at a time, as before. A second `/emergency` gets a private answer with a link to the open one.
- One difference: holders of a bypass role are not held back by an open request. Their `/emergency` pings at once, even while another request is still waiting for votes.
- Bot accounts that join the server are no longer verified automatically, whatever their age. The old bot verified any old-enough account, bots included.
- After any use of `/emergency` the whole server waits 2 minutes before the next, as before.
- Only two kinds of message ever ping the emergency role: the "approved" message and the bypass message. The bot is set up so that no other message it sends can ping a role, even by accident.
- An open request lives in the bot's memory only. If the bot stops or restarts while one is open, the request is closed with the note "cancelled because the bot is restarting. Run /emergency again if it is still needed".
- `/clearemergency` posts its "cancelled" message publicly in the channel, as before.

Verification

- `/verify` answers privately. The old bot answered in the DM, which was private too, so nothing is lost.
- The rules are unchanged: the member must be in the server, must have no role yet, and the account must be at least "/verify age threshold" days old (5 on production). Accounts strictly older than the "Join age threshold" (90 days on production) are verified automatically when they join. The automatic part needs full mode.
- The bot no longer reads DMs. A DM gets the pointer to `/verify`, at most once per 10 minutes per person. The list of who was answered recently is kept in memory only and cleaned out regularly.

/irc

- The answer is visible only to the moderator who ran it. In the old bot it was posted in the channel.
- The mod-log gets a record of every status check and every start, stop or restart that changed something. "Already running" and error answers are not recorded there.

- Errors are explained in plain words: Docker not reachable, the bot's user not allowed to use Docker, container not found. The old bot said nothing in the channel when Docker failed.

Boosts (full mode only)

- Same as before: a new booster gets the booster role and a thank-you DM. Someone who stops boosting is announced in the unboost channel and loses the booster role and any colour roles.
- To notice boosts, and to build the `/boosters` list, the bot loads the full member list when it starts. The list is kept in memory only and is never saved. (An earlier version on the refactor branch did not load it and would have missed most boosts. That is fixed.)

Failures are reported in the mod-log

The service log shows scrambled IDs, so it cannot tell you which member had a problem. Anything a moderator has to act on is therefore posted in the mod-log with the real member mention and the footer "Needs a moderator's attention". Examples:

- the verified role or booster role could not be given, or roles could not be removed after an unboost;
- the verified role or emergency role from the settings no longer exists;
- an emergency was approved but nobody could be pinged, or the request message could not be posted or updated;
- a command failed with an unexpected error.

A problem that would repeat on every join (such as a deleted role) is posted at most once every 10 minutes.

When a command fails unexpectedly, the member is told "The moderators have been notified" only if the mod-log post really went through. Otherwise the member is asked to tell Moderator Mail.

7. Reduced mode, restart protection and logging

Reduced mode

What it is for. It keeps the bot useful while Discord has not (or not yet) approved the Server Members intent. The old bot simply crashed in that situation.

What triggers it. When the bot connects, it asks Discord for the Server Members intent. If Discord refuses, the bot connects exactly one more time without asking for it. There is no further retry.

	Full mode	Reduced mode
`/verify` and every other command	Works	Works
Emergency requests and voting	Works	Works
Answering DMs with the pointer to `/verify`	Works	Works
Automatic verification when someone joins	Works	Off
Boost thank-you and booster role	Works	Off
Removing booster and colour roles after an unboost, and the unboost announcement	Works	Off
`/boosters`	Works	Answers "temporarily unavailable"

What this means in practice while in reduced mode:

- New members with old accounts are not let in automatically. They have to run `/verify` like everyone else. It will work for them.
- Boosts and unboosts that happen during reduced mode are not noticed, and the bot does not catch up on them later. Give or remove the booster role by hand for those members.

The warning. Each time the bot starts in reduced mode it posts one message in the mod-log, titled "Bot started in reduced mode", listing what is off. `/config show` also says which mode the bot is in.

How full mode comes back. The bot only asks Discord at start. So once Discord approves the intent, restart the bot once (section 11). No setting changes.

Restart protection

What a systemd service is. systemd is the part of Linux that starts programs when the server boots and keeps them running. A "service file" (also called a unit file) tells it how: which program, as which user, and what to do when the program stops. The new one is `deploy/verificationbot.service`.

Exit codes. When a program stops, it hands Linux a number saying why. The new bot uses these:

Exit code	Meaning	What the service does
0	Stopped on purpose (you stopped it).	Nothing.
1	Unexpected crash, or the connection to Discord failed for an unknown reason.	Waits 30 seconds, then starts the bot again.
2	`config.json`, `resources/messages.json` or the settings file is missing or broken, or no token is configured.	Never restarts.
3	Discord rejected the token.	Never restarts.
4	Discord refused the connection even in reduced mode.	Never restarts.

The service rules.

- It restarts the bot only after a failure, and waits 30 seconds first.
- It allows at most 5 starts in any hour. After that it stops trying and waits for you.
- It never restarts after exit codes 2, 3 or 4, because starting again cannot fix a bad file, a rejected token or a refused intent.

Why. Discord cancels a bot's token after 1000 connections in 24 hours. The old service restarted every second with no limit and used that up. With the new rules the worst case is 5 starts an hour, each making at most 2 connections. That is at most 240 a day, far below the limit.

Two consequences to know about:

- Starts you do by hand (`systemctl start` or `restart`) count towards the 5 per hour as well. Do not restart the bot over and over while fiddling.
- If the bot does hit the limit (for example the server's internet was down for a long time), it stays stopped until you start it again by hand. The commands are in section 13.

Logging

Where the log is. The bot writes its log lines to "standard output", the normal text output of a program. systemd catches that output and keeps it in the service log. You read it with `journalctl` (commands in section 9). The bot writes no log file of its own.

What is in it. What the bot did and what went wrong, but nothing that identifies a person:

- Every Discord ID (the long 17 to 20 digit numbers) is replaced by a made-up code such as `id:3f9a1c2b7e` before the line is written.
- The same ID gets the same code until the bot is restarted. After a restart all codes are different, and nobody can turn a code back into an ID.
- No usernames, no DM text and no emergency reasons are ever logged.

An example line (the code is made up):

```
2026-10-12 21:04:11 | INFO     | verification_bot.cogs.verification | Member id:3f9a1c2b7e verified via
      /verify; account age 412 days
```

When you need to know WHO. Look in the mod-log channel in Discord. Every event that matters (verified, refused, auto-verified, emergency requested, each vote, outcomes, `/irc` actions, setting changes, failures) is posted there with the real member, name and avatar. Use the time of the log line to find the matching mod-log entry.

8. Test checklist for the test server

What this is for

Automated tests cannot click buttons in Discord. Only you can prove that the commands really work. You run the list below twice in your test server, with your test bot:

- **Run A, full mode:** the test bot's Server Members switch is ON in the Developer Portal.
- **Run B, reduced mode:** the same switch is OFF.

The Presence and Message Content switches stay OFF both times. The switches are at https://discord.com/developers/applications > your TEST bot > Bot > Privileged Gateway Intents. Remember to press Save. At the time of writing, the test bot's Server Members switch is OFF.

What you need

- Your PC (the WSL terminal), in the folder `/home/massimo/verification-bot`.
- Two Discord accounts in the test server. Account 1 is your own (administrator). Account 2 is a plain second account.
- Account 1 needs the verified role (to call an emergency) and also some role that sits above the verified role in the role list (to be allowed to vote).
- The test bot's own role must sit above the verified role in Server Settings > Roles, or it cannot hand the role out.
- A throwaway Docker container for the `/irc` test.

Each start of the bot uses 1 connection to Discord (2 when it falls back to reduced mode). The limit is 1000 a day, so there is plenty of room, but there is no reason to restart more than needed.

Prepare (once)

A **virtual environment** is a private folder (here `.venv`) holding the Python packages this bot needs, so they do not mix with the rest of the system. Create it and install the packages:

```
cd /home/massimo/verification-bot
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
```

Create the throwaway container for `/irc`:

```
docker run -d --name verification-bot-irc-test alpine sleep 3600
```

Then open `config.test.json` in an editor and set `"irc_relay_id"` to `"verification-bot-irc-test"`. Do this before starting the bot, because `config.test.json` is only read at start. (The container stops by itself after an hour. `docker start verification-bot-irc-test` brings it back.)

Start the test bot

```
cd /home/massimo/verification-bot
.venv/bin/python verif.py --config config.test.json --settings data/settings.test.json
```

`--config` tells the bot to use the test server's file instead of the production one. `--settings` tells it to keep the test settings in their own file, so they can never mix with production settings. The bot keeps running in that terminal and prints its log there. Stop it with Ctrl+C.

What the log should say

In both runs, near the top (the first line appears only on the very first start, when the settings file does not exist yet):

```
... | Imported 17 setting(s) from config.json into data/settings.test.json
... | Loaded extension verification_bot.cogs.verification
    (five more "Loaded extension" lines)
... | Registered 9 server command(s) and 3 global command(s)
```

Run A (full mode) then continues with:

```
... | Logged in as bot id:xxxxxxxxxx
... | Running in FULL mode: Server Members intent granted, N member(s) loaded
```

Run B (reduced mode) continues with:

```
... | WARNING | ... | Discord refused the Server Members intent. Starting once more without it (reduced
      mode).
    (the six "Loaded extension" lines again)
... | Logged in as bot id:xxxxxxxxxx
... | WARNING | ... | Running in REDUCED mode: Discord refused the Server Members intent. Auto-verify on
      join, boost tracking and /boosters are off.
```

Two warnings about voice support ("PyNaCl is not installed", "davey is not installed") are normal. The bot does not use voice.

Step 0: fix the test settings (first run only)

The test config contains three production values that do not exist in the test server. This is expected.

- ☐ Run `/config show`. Expected: a private answer listing all 21 settings. It says "Mode: full" in run A and "Mode: reduced" in run B. It marks at least these three with "Needs attention": **On-topic category**, **Emergency bypass roles**, **Unboost channel**. Rules, Roles and Booster channel are also marked, because they are not set in a test server.
- ☐ Fix the category: `/config category setting:On-topic category category:(a category in the test server)`. Expected: private answer with Before and After.

- ☐ Fix the bypass list: `/config role-prune setting:Emergency bypass roles`. Expected: "1 deleted role dropped from the list."
- ☐ Fix the unboost channel: `/config channel setting:Unboost channel channel:(any text channel)`.
- ☐ Optional: set Rules channel, Roles channel and Booster channel the same way.
- ☐ Look at the mod-log channel. Expected: one "Setting changed" entry per change, with your name, the value before and the value after.
- ☐ Run `/config show` again. Expected: the three items are no longer marked. Check that Verified role, Emergency role and Mod-log channel show real roles and a real channel of the test server.
- ☐ Make the emergency tests quick: `/config number setting:Emergency approval threshold value:1` and `/config number setting:Emergency denial threshold value:1`.
- ☐ With account 2 (not an administrator), type `/config`. Expected: the command is not offered at all.

The checklist (do it in run A and again in run B)

Verification

- ☐ Make sure account 2 has no roles in the test server (remove them by hand if needed).
- ☐ Account 2 sends the bot a DM with any text. Expected: one reply pointing to `/verify`. Send a second DM straight away. Expected: no second reply (10 minute pause).
- ☐ Account 2 runs `/verify` in the DM with the bot. Expected: a private "You have been verified" answer, account 2 now has the verified role, and the mod-log shows "Member verified".
- ☐ Account 2 runs `/verify` again, this time in a channel of the server. Expected: a private "You already have access to the server".
- ☐ Optional: remove the role from account 2, set `/config number setting:/verify age threshold (days) value:36500`, and run `/verify`. Expected: a private "your Discord account must be at least 36500 days old" and a "Verification refused" entry in the mod-log. Set the number back to what it was afterwards (`/config show` showed it; 5 is the default).

Help and privacy

- ☐ `/help` with account 2. Expected: a private list with the sections Everyone, Verified members, Moderators, Administrators. `/config` subcommands are under Administrators, `/irc` and `/clearemergency` under Moderators.
- ☐ `/privacy`. Expected: a private summary, a line on how to ask for deletion, and the link to `PRIVACY.md`.
- ☐ Run `/help` in a DM with the bot. Expected: it works there too.

Small commands (in a server channel)

- ☐ `/ping`. Expected: "Pong" with a latency in milliseconds.
- ☐ `/uptime`. Expected: how long the bot has been running and since when.
- ☐ `/github`. Expected: the link to the source code.
- ☐ `/steve`. Expected: a small coloured box. The Steve emoji belong to the production server, so in the test server they may show as text such as `:steve1:`. That is fine.
- ☐ Run `/ping` twice quickly. Expected the second time: "That command is on cooldown."

Boosters

- ☐ `/boosters`. Expected in run A: a public "Server boosters" answer with the number of boosts (probably 0). Expected in run B: a private "The booster list is temporarily unavailable."

Emergency (use a channel inside the on-topic category; both accounts need the verified role; wait 2 minutes between two `/emergency` commands, or you get "on cooldown")

- ☐ Account 2 runs `/emergency` in a channel outside the category. Expected: a private "can only be used in channels under ... category".
- ☐ Wrong buttons: account 1 runs `/emergency reason:testing the buttons, please ignore`. Expected: a private confirmation for account 1, and a normal message from the bot with the reason, "Approved: 0/1", "Denied: 0/1" and two buttons. The mod-log shows "Emergency requested" with the reason.
- ☐ Account 1 (the caller) presses Approve. Expected: a private "You cannot vote on your own emergency request."
- ☐ Account 2 (only the verified role) presses Approve. Expected: a private "Only members of sudo level 1 or higher can vote".
- ☐ Cancel: account 1 runs `/clearemergency`. Expected: a public "cancelled" message, the request message now says "This request is closed: cancelled by a moderator" and has no buttons, and the mod-log shows who cancelled it.
- ☐ Approve path: account 2 runs `/emergency` with a reason. Account 1 presses Approve. Expected: a private "Your vote to approve has been recorded", then a new message that pings the emergency role, the request message closed as "approved" with no buttons, and mod-log entries for the vote and the outcome.
- ☐ Deny path: account 2 runs `/emergency` again. Account 1 presses Deny. Expected: a message telling account 2 the request was denied, no role ping, request closed as "denied" with no buttons.
- ☐ Timeout: run `/config number setting:Emergency reminder interval (seconds) value:15` and `/config number setting:Emergency reminder count value:1`. Account 2 runs `/emergency`. Press nothing. Expected: a reminder after 15 seconds, and after 30 seconds a "timed out" message, with the request closed and its buttons gone.
- ☐ Recommended once: set the interval back to 180 and the count to 5, open a request and leave it for the full 18 minutes. Expected: five reminders, then the timeout, and the buttons disappear. This is the case that was broken on the refactor branch.
- ☐ Bypass: give account 1 any spare role, then `/config role-add setting:Emergency bypass roles role:(that role)`. Account 1 runs `/emergency` with a reason. Expected: no vote. The bot at once posts a message that pings the emergency role, and the mod-log shows "Emergency called (approval bypassed)".
- ☐ Voter roles: `/config role-add setting:Sudo voter roles role:(a role account 1 does not have)`. Account 2 opens a request and account 1 presses Approve. Expected: account 1 is now refused as "not sudo level 1 or higher". Remove the role from the list again with `/config role-remove`, then `/clearemergency`.

IRC relay (account 1; the throwaway container must exist)

- ☐ `/irc action:status`. Expected: a private answer "The IRC relay is `running`" and the same record in the mod-log.
- ☐ `/irc action:stop`. Expected: "The IRC relay was stopped. It is now `exited`." (This can take about 10 seconds.)
- ☐ `/irc action:stop` again. Expected: "The IRC relay is already stopped". No new mod-log entry.
- ☐ `/irc action:start`, then `/irc action:restart`. Expected: "started" and "restarted", each "now `running`".

- ☐ With account 2, type `/irc`. Expected: not offered, or refused with "You do not have permission to use this command."
- ☐ If you have a moderator who is not an administrator: check that they see `/irc` and `/clearemergency` in the menu (see the note in section 2).

A new member joining (run A only)

- ☐ Account 2 leaves the test server and joins again. Expected, if the account is older than the join age threshold: it gets the verified role by itself within a second or two, and the mod-log shows "Member auto-verified". In run B nothing happens on join, and that is correct.

Stopping

- ☐ Open an emergency request, then press Ctrl+C in the terminal. Expected: the log says "Shutdown requested; closing the connection", the program ends within a few seconds, and the open request is closed with "cancelled because the bot is restarting".
- ☐ Start the bot again for the next run only after changing the Server Members switch.

Run B only: the warning

- ☐ Right after the start, look at the mod-log. Expected: one message "Bot started in reduced mode" that lists what is off.

Clean up

```
docker rm -f verification-bot-irc-test
```

What cannot be tested there

- **Nitro boost events**: the thank-you DM, the booster role, the unboost announcement and the removal of booster and colour roles. They only happen when someone really boosts or stops boosting. They are covered by automated tests only.
- **The production server itself**: the service file, Python on `cannizzaro`, and the bot's access to Docker there. You check those while deploying (section 9).

9. Deploying on the server

These commands are for the server `cannizzaro`, logged in as `massimo`. Do them in order, after pull request #36 is merged into `master`, and after you have the new token (section 10, step 1). Lines starting with `#` are comments.

Step 1: stop the old bot for good

This makes sure the old service is not running and will not start at boot.

```
sudo systemctl disable --now verificationbot
```

Step 2: get the new code

```
cd /home/massimo/verification-bot
git checkout master
git pull
```

The old bot left a log file with usernames and DM text in this folder. The new bot does not use it. Delete it:

```
rm -f verification.log
```

Step 3: create the virtual environment and install the packages

The virtual environment (`.venv`) is a private folder with the Python packages the bot needs. The old bot used packages installed in your home folder (`~/.local`). The new service uses `.venv` instead, so that a system update cannot break the bot.

```
cd /home/massimo/verification-bot
python3 -m venv .venv
.venv/bin/pip install --upgrade pip
.venv/bin/pip install -r requirements.txt
```

This installs two main packages (plus the smaller ones they depend on): `discord.py` (talks to Discord) and `docker` (talks to Docker, for `/irc`). The server has Python 3.9, and the bot supports it.

Step 4: put the NEW token in config.json

The token is the bot's password for Discord. Discord cancelled the old one when the bot looped, so the old one will never work again. The new one comes from the Developer Portal (section 10, step 1).

```
nano config.json
```

Replace the value after `"token":` with the new token. Keep the quotation marks around it. Change nothing else: the rest of the file is what the one-time import reads. Save with Ctrl+O, Enter, then leave with Ctrl+X.

Make the file readable by you only:

```
chmod 600 config.json
```

Step 5: make sure the bot can talk to Docker (for /irc)

`/irc` starts and stops the IRC relay container by talking to Docker on the server. The bot runs as user `massimo`, so `massimo` must be allowed to use Docker. Test it (no `sudo`):

```
docker ps --format '{{.Names}}'
```

- If this prints a list of container names, all is well.
- If it says "permission denied", add `massimo` to the `docker` group, then log out and in again:

```
sudo usermod -aG docker massimo
```

The list also shows you the **name** of the IRC relay container. Put that name in `irc_relay_id` in `config.json`, in place of the long ID. A container's ID changes every time the container is recreated (for example after an update). Its name stays the same.

Step 6: install the new service file

Unit files can live in two places. The old one is in `/usr/lib/systemd/system/`. The new one goes into `/etc/systemd/system/`, the place meant for files an administrator adds. When both places hold a file with the same name, systemd uses the one in `/etc`. Removing the old one anyway avoids any confusion later.

```
cd /home/massimo/verification-bot
sudo cp deploy/verificationbot.service /etc/systemd/system/verificationbot.service
sudo rm /usr/lib/systemd/system/verificationbot.service
sudo systemctl daemon-reload
```

`daemon-reload` tells systemd to read the unit files again. Check that it picked up the new one:

```
systemctl cat verificationbot
```

The first line of the output must show the path `/etc/systemd/system/verificationbot.service`, and further down you must see `Restart=on-failure` and `RestartSec=30`. If you still see `Restart=always`, stop here and do not start the bot.

Step 7: start the bot and watch the log

`enable` makes the bot start when the server boots. `--now` also starts it right away.

```
sudo systemctl enable --now verificationbot
journalctl -u verificationbot -f
```

`journalctl -u verificationbot -f` shows the service log and keeps following it. Leave it with Ctrl+C (this does not stop the bot).

What a good start looks like:

```
... | Imported 20 setting(s) from config.json into data/settings.json
... | Loaded extension verification_bot.cogs.verification
    (five more "Loaded extension" lines)
... | Registered 9 server command(s) and 3 global command(s)
... | Logged in as bot id:xxxxxxxxxx
... | Running in FULL mode: Server Members intent granted, N member(s) loaded
```

N should be close to the size of the server (about 25,000). Or, if Discord has not approved the intent yet, which is the likely case on the first day:

```
... | Registered 9 server command(s) and 3 global command(s)
... | WARNING | ... | Discord refused the Server Members intent. Starting once more without it (reduced
      mode).
... | Logged in as bot id:xxxxxxxxxx
... | WARNING | ... | Running in REDUCED mode: Discord refused the Server Members intent. Auto-verify on
      join, boost tracking and /boosters are off.
```

Both are good starts. In the second case you will also find the "Bot started in reduced mode" message in the mod-log. If the log shows anything else, go to section 13.

Other useful commands:

```
systemctl status verificationbot             # is it running?
journalctl -u verificationbot -n 100         # the last 100 log lines
```

```
journalctl -u verificationbot --since today   # everything since midnight
sudo systemctl restart verificationbot        # stop and start (counts towards 5 per hour)
sudo systemctl stop verificationbot           # stop
```

Step 8: check the imported settings

In the /r/sysadmin server, run:

```
/config show
```

Expected: all 21 settings with your production values, and ideally "All settings look good". Read through it once. Pay attention to:

- **Emergency bypass roles**: the role listed here can now ping without a vote (section 6).
- **Sudo voter roles**: empty at first. That keeps the old voting rule. Add your sudo roles with `/config role-add` when you want voting limited to them.
- **Unboost channel, Rules channel, Roles channel, Booster channel**: check they are the channels you expect.
- Anything marked "Needs attention".

Then try `/verify` with a second account, or ask a new member to.

How to go back

If something is badly wrong, stop the bot first:

```
sudo systemctl stop verificationbot
```

The old code is still in git: it is `master` as it was before the merge. You can see the history with `git log --oneline` and return to an older state with `git checkout` followed by the commit's code.

Be aware that going back does not bring the old bot back to life. The old bot needs the Message Content intent to read "verify" in DMs and asks Discord for all intents at start. Discord has removed them, so the old bot cannot connect. And never reinstall the old service file: it is the one that restarted every second. In practice the way out of a problem is to fix it going forward. The new bot's worst case is that it stops and waits for you.

10. Developer Portal steps

The Developer Portal is Discord's website for bot owners: https://discord.com/developers/applications. Pick the PRODUCTION bot (not the test bot).

Step 1: get a new token

1. Open the **Bot** page in the left menu.
2. Press **Reset Token** and confirm. Discord may ask for your two-factor code.
3. Press **Copy**. The token is shown only once. If you lose it, reset it again.
4. Put it in `config.json` on the server (section 9, step 4). Do not paste it anywhere else: not in a chat, not in git, not in a screenshot.

Step 2: switch off the intents the bot no longer uses

On the same Bot page, under **Privileged Gateway Intents**:

- **Presence Intent**: OFF.
- **Message Content Intent**: OFF.
- **Server Members Intent**: leave it as it is. For a bot of this size Discord decides this one through the review, not you. If Discord has already removed it, the switch may be off or locked. That is expected, and the bot handles it by starting in reduced mode.

Press **Save Changes** if the page offers it. If the Presence and Message Content switches are already off or locked, there is nothing to do.

Step 3: set the privacy policy link

Discord requires every bot to have a privacy policy linked in the portal. Open **General Information** and put this in **Privacy Policy URL**:

```
https://github.com/mmcmd/verification-bot/blob/master/PRIVACY.md
```

The link works once pull request #36 is merged, because that puts the new `PRIVACY.md` on `master`. The `/privacy` command shows the same link.

Step 4: submit the application

1. Open `docs/discord-intent-application.md`. It contains ready-to-paste answers for each field of Discord's form.
2. Add your screenshot links where that file has marked slots. It says what each screenshot should show.
3. In the portal, start the privileged intent review for the bot and paste the answers.
4. Tick **Server Members only**. Leave Presence and Message Content unticked. The form warns that unticked intents are lost when the application is reviewed. That is what we want.
5. Answer "No" to the question about storing data outside Discord, as the file says.

The list of form fields was read from the portal's program code, not from the form itself, so the real form may be worded or ordered a little differently. Where exactly the form opens in the portal could not be checked either.

Discord's own pages on this:

- Announcement: https://support-dev.discord.com/hc/en-us/articles/40281523410967-Changes-to-Privileged-Intent-Access-for-Discord-Apps
- Review guide: https://docs.discord.com/developers/gateway/getting-started-with-privileged-intent-review

11. When Discord approves

The bot asks for the intent only when it starts. So after the approval, restart it once:

```
sudo systemctl restart verificationbot
journalctl -u verificationbot -n 30
```

Check three things:

- ☐ The log says `Running in FULL mode: Server Members intent granted, ... member(s) loaded`, with a member count close to the size of the server.
- ☐ `/boosters` shows the list again.
- ☐ `/config show` says "Mode: full", and no new "Bot started in reduced mode" message appeared in the mod-log.

No setting needs changing. Boosts that happened while the bot was in reduced mode are not caught up, so give those members the booster role by hand.

If the log still says REDUCED mode, the approval has not taken effect yet. Wait and try again later. Do not restart again and again.

The review repeats every year. Missing the window is what started this outage. Put a reminder in your calendar for about 10 months after the approval date: "Re-apply for the verification bot's Server Members intent". Keep `docs/discord-intent-application.md`, because you can reuse the answers.

If Discord says no, the bot keeps running in reduced mode for as long as needed. Members can still verify with `/verify`.

12. What was tested and how

Automated tests. 251 tests pass on Python 3.9.25 (the version on the server) and on Python 3.12, and the code checker `ruff` reports nothing. These tests run the bot's code against stand-ins for Discord. They never connect to Discord. They cover:

- the import of an old-style `config.json` with the same shape as the production file (text values, the single bypass value, the three production channels, the replaced status text, the ignored keys);
- the settings file: saving, reloading, bad values, a broken file;
- start-up: one connection in full mode, exactly one more in reduced mode, the right exit code for a bad config, a rejected token and a crash, and that the exit codes are the ones listed in the service file;
- `/verify`, automatic verification on join, the DM pointer and its 10 minute pause;
- the emergency vote: thresholds, who may vote, the bypass, reminders and timeout timing, button removal, which messages ping the role;
- boosts, unboosts, and `/boosters` in both modes;
- `/help`, `/privacy`, `/config` (including that non-administrators are refused), `/irc` against a stand-in for Docker;
- the log: IDs are scrambled, also inside error traces.

Proven live, with the test bot.

- One real start with the test bot, whose Server Members switch was OFF: it logged in, Discord refused the intent, the bot started itself once more in reduced mode, imported 17 settings from the test config, registered 9 server commands and 3 global commands, and stopped cleanly on Ctrl+C.
- The `/irc` code was run against a real throwaway Docker container: status, stop, start and restart, by name and by ID. This was done from a script, not by clicking the command in Discord.

Not proven. Only your click-through can show these.

- Any slash command or button actually being clicked in Discord.
- Full mode with a real connection (the test bot's switch was off).
- Boost and unboost events (not testable without a real boost).
- Everything on the production server: the service file under systemd, the virtual environment on that machine, Docker access for user `massimo`, and the import of the real production `config.json`.

13. Troubleshooting

What you see	What it means	What to do
Service stopped, log says "Configuration error: ..." (exit code 2)	`config.json`, `resources/messages.json` or `data/settings.json` is missing, is not valid JSON (often a missing comma or quotation mark), or the bot cannot write to the `data` folder. The message names the file.	Fix the file. Then `sudo systemctl start verificationbot`. The service will not retry by itself.
Service stopped, log says "Discord rejected the bot token" (exit code 3)	The token in `config.json` is wrong, old, or was pasted with a missing character. A `DISCORD_TOKEN` environment variable, if set, wins over the file.	Reset the token in the Developer Portal, paste the new one, then `sudo systemctl start verificationbot`. Do not start it repeatedly with the same token.
Service stopped, log says "Discord refused the connection even without the Server Members intent" or "refused the gateway intents" (exit code 4)	Discord refused the bot even in reduced mode. This should not happen, since reduced mode asks for no privileged intent.	Check the Bot page in the Developer Portal for a notice from Discord. Do not keep restarting.
`systemctl status` shows "start-limit-hit", or `start` answers "Start request repeated too quickly"	The bot was started 5 times within an hour, so systemd stopped trying.	Read the log first (`journalctl -u verificationbot -n 100`) and fix the cause. Then `sudo systemctl reset-failed verificationbot` followed by `sudo systemctl start verificationbot`.
`systemctl status` shows "status=203/EXEC"	systemd could not run `.venv/bin/python`. Usually the `.venv` folder is missing.	Redo section 9, step 3. If the folder exists and the error stays, the server's security rules may block running programs from a home folder; the log line from systemd will say "Permission denied".
Slash commands do not show up in Discord	Several possible causes.	1. Check the log for "Registered 9 server command(s) and 3 global command(s)". 2. Restart your Discord app (Ctrl+R on desktop). 3. If the log says "Could not register commands in the home server", the bot was invited long ago without the permission to create slash commands. Invite it again with a link containing `scope=bot+applications.commands` (you do not need to remove it first), then restart the bot once. 4. `/config` is hidden from members who are not administrators.
Only 8 server commands registered, no `/irc`	`irc_relay_id` in `config.json` is empty or missing.	Put the container name there and restart the bot.
`/irc` says "could not reach the Docker daemon"	Docker is not running, or user `massimo` may not use it.	Section 9, step 5. After adding the group, restart the bot.
`/irc` says "No Docker container named or numbered ..."	`irc_relay_id` does not match any container. The container was probably recreated and has a new ID.	Put the container's name in `irc_relay_id` and restart the bot.

What you see	What it means	What to do
`/irc` says "Docker refused or failed the ... action"	Docker itself reported an error.	Look at the service log around that time, and at `docker ps -a`.
"Bot started in reduced mode" appears in the mod-log	Discord did not allow the Server Members intent at this start.	Nothing urgent. `/verify` works. Submit the application (section 10). After approval, restart once (section 11).
A command answers "not set up yet ... An administrator has to run /config"	A setting that command needs has no value.	`/config show`, then set what is marked.
A member says "Something went wrong while running that command"	An unexpected error.	Look in the mod-log for the entry with the footer "Needs a moderator's attention". It names the member, the command and the error.
Mod-log stays empty	The Mod-log channel is not set, was deleted, or the bot lacks View Channel, Send Messages or Embed Links there.	`/config show` says which. Fix the permission or choose the channel again with `/config channel`.
The bot gives nobody the verified role, mod-log says "Could not assign the verified role"	The bot lacks Manage Roles, or its own role is below the verified role.	In Server Settings > Roles, drag the bot's role above the verified role.

Other files to read: `README.md` (short overview and setup), `PRIVACY.md` (the privacy policy members see), `docs/discord-intent-application.md` (the answers for Discord's form).